2025 IEEE International Symposium on High Performance Computer Architecture (HPCA) | 979-8-3315-0647-6/25/$31.00 ©2025 IEEE | DOI: 10.1109/HPCA61900.2025.00065

NeuVSA: A Unified and Efficient Accelerator for Neural Vector Search

Ziming Yuan[1,2], Lei Dai[1,2], Wen Li[3], Jie Zhang[4,5], Shengwen Liang[1,5], Ying Wang[1], Cheng Liu[1], Huawei Li[1,2], Xiaowei Li[1,2,5], Jiafeng Guo[6,2], Peng Wang[7], Renhai Chen[7], and Gong Zhang[7]
[1]State Key Lab of Processors, Institute of Computing Technology, CAS.
[2]University of Chinese Academy of Sciences.
[3]School of Computer and Information Technology, Shanxi University.
[4]School of Computer Science, Peking University. [5]Zhongguancun National Laboratory, Beijing.
[6]Key Lab of Network Data Science and Technology, Institute of Computing Technology, CAS.
[7]Huawei Technologies Co., Ltd. China.
Email(s): {yuanziming22s, dailei19z}@ict.ac.cn, liwen@sxu.edu.cn, jiez@pku.edu.cn,
{liangshengwen, wangying2009, liucheng, lihuawei, lxw, guojiafeng}@ict.ac.cn,
wangpeng423@huawei.com, renhai.chen@tju.edu.cn, nicholas.zhang@huawei.com

***Abstract*—Neural Vector Search (NVS) has exhibited superior search quality over traditional key-based strategies for information retrieval tasks. An effective NVS architecture requires high recall, low latency, and high throughput to enhance user experience and cost-efficiency. However, implementing NVS on existing neural network accelerators and vector search accelerators is sub-optimal due to the separation between the embedding stage and vector search stage at both algorithm and architecture levels. Fortunately, we unveil that Product Quantization (PQ) opens up an opportunity to break separation. However, existing PQ algorithms and accelerators still focus on either the embedding stage or the vector search stage, rather than both simultaneously. Simply combining existing solutions still follows the beaten track of separation and suffers from insufficient parallelization, frequent data access conflicts, and the absence of scheduling, thus failing to reach optimal recall, latency, and throughput.**

To this end, we propose a unified and efficient NVS accelerator dubbed NeuVSA based on algorithm and architecture co-design philosophy. Specifically, *on the algorithm level*, we propose a learned PQ-based unified NVS algorithm that consolidates two separate stages into the same computing and memory access paradigm. It integrates an end-to-end joint training strategy to learn the optimal codebook and index for enhanced recall and reduced PQ complexity, thus achieving smoother acceleration. *On the architecture level*, we customize a homogeneous NVS accelerator based on the unified NVS algorithm. Each sub-accelerator is optimized to exploit all parallelism exposed by unified NVS, incorporating a structured index assignment strategy and an elastic on-chip buffer to alleviate buffer conflicts for reduced latency. All sub-accelerators are coordinated using a hardware-aware scheduling strategy for boosted throughput. Experimental results show that the joint training strategy improves recall by 4.6% over the separated strategy and accuracy by 43.5% over LUT-NN. NeuVSA achieves 2.82× to 416.17× lower latency over CPU, GPU, DFX+ANNA, and PQA+ANNA, and up to 49.60× and 10.57× higher average throughput over CPU and GPU, respectively. NeuVSA also reduces chip area by 65.2% over PQA+ANNA.

I. Introduction

Neural Vector Search (NVS) has gained increasing attention in information retrieval systems as it revolutionizes the conventional keyword-based search paradigm by understanding the context and intent of user queries through neural networks [21], [33], [41], [52]. Internet giants such as Google (Vertex AI [17]), Meta (Faiss [25]), and Alibaba (Proxima [3]) have used this technique to deliver more prompt responses.

NVS involves two primary stages: ***embedding*** and ***vector search***. The embedding stage utilizes embedding models (e.g., item2vec [7], graph2vec [18], and BERT [28]) to project the semantics of objects to high-dimensional vectors. The vector search stage retrieves the relevant objects for a given query using generated vectors. A practical NVS architecture aims to achieve high recall, low latency, and high throughput because high recall enhances ranking efficiency for more accurate responses, low latency reduces user wait time, and high throughput helps keep serving cost tractable.

However, none of the existing neural network accelerators [19], [49] and vector search accelerators [29] can satisfy the demand of the practical NVS architecture due to the separation between the embedding stage and the vector search stage at the algorithm and architecture levels. Specifically, at the algorithm level, two stages are trained separately and therefore may not be optimally compatible, resulting in decayed recall, as they use task-independent reconstruction error as the loss function [61], Our experiment shows that joint training can increase recall by approximately 5% over the separation strategy. On the architecture level, existing accelerators focus on either the embedding stage or the vector search stage rather than both simultaneously. Simply gluing the existing accelerators not only lacks a suitable scheduling policy but also suffers from (1) ***long latency*** because vector data have to transfer from one accelerator to another (Fig. 1), (2) ***high energy, area, and cost*** as extra hardware resources (e.g. router) are required to support vector transfer, and (3) ***low hardware utilization***, due to accelerators designed for two stage have different architectures and cannot achieve resource sharing. Our experiment shows that hardware utilization is not up to 50% when deploying NVS on DFX [19]+ANNA [29].

Building a unified NVS at both the algorithm and architec-

979-8-3315-0647-6/25/$31.00 ©2025 IEEE
DOI 10.1109/HPCA61900.2025.00065

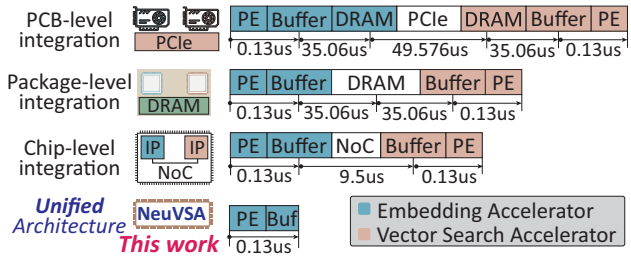

Fig. 1: The transfer latency of 8KB data using various integration methods.

ture levels is a promising solution to breaking the separation wall. Fortunately, we observe that Product Quantization (PQ) opens up an opportunity as it is not only a popular technique for vector search but also can substitute general matrix multiply (GEMM) operations in the embedding stage with lookup-table operations. Regrettably, current PQ-based algorithms and accelerators still focus on either the embedding stage (e.g., PQA [16]) or the vector search stage (e.g., ANNA [29]), rather than both simultaneously. The simple combination of existing PQ-based solutions (e.g., PQA+ANNA) still follows the beaten track of separation and cannot satisfy the demand of NVS due to the absence of joint training and the difference in algorithm details. For example, PQA [16] employs symmetric distance computation (SDC), while ANNA adopts asymmetric distance computation (ADC). Even worse, existing PQ-based accelerators are suboptimal because (1) ***Insufficient parallelization.*** PQ-based unified NVS offers six levels of parallelism, whereas existing accelerators can exploit up to four levels, resulting in under-utilization and high latency. (2) ***Frequent buffer access conflicts.*** The random distribution of the index used by the lookup table can lead to frequent data access conflicts, reducing the utilization of the existing PQ accelerator to less than 50%. (3) ***Lack of scheduling.*** Two stages in NVS exhibit distinct hardware resource demands for different workloads. However, the scheduling strategy provided by existing PQ accelerators focuses only on a single stage, lacking the scheduling mechanism required by NVS.

To break the separation wall and overcome the issues of existing PQ solutions, we adopt an algorithm and architecture co-design philosophy to customize an NVS acceleration solution. Specifically, our contributions are as follows:

- We propose a unified and efficient NVS framework with accelerator dubbed NeuVSA, which unifies two separate stages into the same computing and memory access paradigm at the algorithm and architecture levels. Such a unified design can simplify the difficulty of joint training for high recall, reduce the complexity of the architecture for reduced latency, and tackle the challenge of workload scheduling for boosted throughput.
- On the algorithm level, we propose a learned PQ-based unified NVS algorithm that abstracts NVS into two main primitives: the product operator and the indexing operator. This algorithm incorporates an end-to-end joint training strategy with distortion-ranking loss and codebook learning method to learn the optimal codebook and index for enhanced recall and reduced memory footprint, pushing the proposed unified NVS algorithm towards smoother hardware acceleration.
- On the architecture level, we customize a homogeneous NVS accelerator based on the proposed unified NVS algorithm. Each sub-accelerator boosts the product operator and the indexing operator by harnessing all parallelism exposed by unified NVS. We design a structured index assignment strategy and an elastic LUT buffer to alleviate buffer conflicts for reduced latency. All sub-accelerators are coordinated using a hardware-aware scheduling strategy to improve throughput by considering the features of workloads and targeted accelerators.
- Experiments on typical NVS applications validate the advantages of NeuVSA. The joint training strategy improves recall by 4.6% over the separated strategy and accuracy by 43.5% over LUT-NN. NeuVSA achieves 2.82× to 416.17× lower latency over CPU, GPU, DFX+ANNA and PQA+ANNA, and up to 49.60× and 10.57× higher average throughput over CPU and GPU solutions, respectively. NeuVSA also reduces the chip area by 65.2% compared to PQA+ANNA.

II. Background

A. Neural vector search

Neural vector search aims to utilize deep learning algorithms to identify the semantic similarities between queries and candidate objects, which consists of two stages:

Embedding stage aims to map the semantic meaning of objects to high-dimensional vectors. Formally, given an object $\mathcal{O}$ with versatile semantics, the embedding stage generates a vector $X = [x_0, x_1, ..., x_{D-1}] \in \mathbb{R}^D$ in a D dimensional space using the embedding function $\varphi(\mathcal{O})$. X is formulated as:

$$X = \varphi(\mathcal{O}) : \mathcal{O} \rightarrow X \tag{1}$$

Vector Search stage focuses on efficiently retrieving the K closest neighbor in a high-dimensional space for a query vector. Formally, given a dataset with n instances $Y = \{y_0, ..., y_{n-1}\}$, each instance $y_i \in Y$ is embedded to a vector $X = [x_0, x_1, ..., x_{D-1}] \in \mathbb{R}^D$ in the embedding stage, where D is the dimension. Given a query q and its vector $X_q \in \mathbb{R}^D$ and a value k, the vector search obtains k nearest neighbors Ψ in Y to q by evaluating $\phi(x, x_q)$, where $\phi(x, x_q)$ is a user-defined distance computing function. Ψ is described as follows:

$$\Psi = \underset{\Psi \subseteq Y, |\Psi| = k}{\operatorname{argmin}} \sum_{x \in \Psi} \phi(x, x_q) \tag{2}$$

Since exhaustive vector search suffers from high complexity, approximate vector search gains more attention which relies on four primary methods: LSH-based [34], [47], tree-based [20], [38], graph-based [51], [66], and quantization-based [39], [59]. Among these methods, the quantization-based approach (such as Product Quantization (PQ)) is the most popular choice for large-scale search scenarios [29].

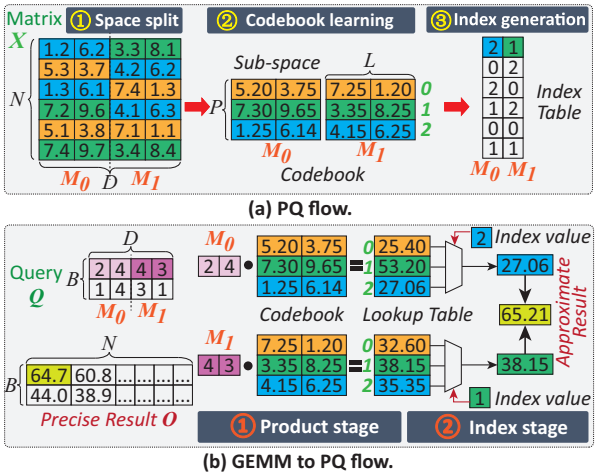

Fig. 2: (a) Illustration of Product Quantization. Each color represents a sub-vector. The sub-vector with the same color represents that they belong to the same cluster. (b) PQ for approximated GEMM. Result $O = QX^T$, $\cdot$ is a dot operator symbol with each row.

B. Product quantization (PQ)

PQ aims to assign each value in a matrix as an element of a finite set (the codebook) to compress the original tensor. Fig. 2a depicts this process, which includes three steps:

- **Space split①**. The matrix $X \in \mathbb{R}^{N \times D}$ is divided into the M submatrices $X_j \in \mathbb{R}^{N \times L} (0 \leq j \leq M-1)$, where $L=D/M$.
- **Codebook learning②**. For each submatrix $X_m \in \mathbb{R}^{N \times L}$, PQ aims to learn a codebook $\mathcal{C} \in \mathbb{R}^{P \times L} (P \ll N)$ that contains P codewords $c(p) \in \mathbb{R}^L, p = \{0, 1, ..., P-1\}$ using k-means [45], which minimizes the distance sum of each submatrix X_m^n and its closest codeword $c(p)$.
- **Index assignment③**. Each submatrix $X_m \in \mathbb{R}^{N \times L}$ is encoded to the index $\mathcal{I} \in \{0,1\}^{N \times P}$ based on $\mathcal{C} = \{c_0, ..., c_{p-1}\}$, where each row only contains one nonzero entry that is used to locate one nearest value in $\mathcal{C}$. Each row in $\mathcal{I}$ can be replaced in binary form using $log_2(P)$ bits for storage efficiency.

C. PQ for approximated matrix multiplication

Based on the codebook $\mathcal{C}$ and index table $\mathcal{I}$ in Section. II-B, PQ can be used for approximated general matrix multiplication (GEMM) by replacing partial multiplication operations with lookup table operations. Specifically, as shown in Fig. 2b, the query $Q \in \mathbb{R}^{B \times D}$ is multiplied by the matrix $X^T \in \mathbb{R}^{D \times N}$ to generate the output $O = QX^T$, where $O \in \mathbb{R}^{B \times N}$. Mapping PQ to GEMM requires dividing Q into M subspaces along with D oriented. The m-th subspace of Q is $Q_m \in \mathbb{R}^{B \times L}$, where $L=D/M$. PQ-based GEMM includes two stages:

- **Product stage**: Each subspace of query $Q_m \in \mathbb{R}^{B \times L}$ is multiplied by codewords in the codebook to generate intermediate result $\mathcal{IR} \in \mathbb{R}^{B \times M \times P}$. This operation reduces the number of multiplications in GEMM from $B \times N \times D$ to $B \times P \times M \times L$ in PQ, a decrease of a factor of $\frac{N}{P}$.

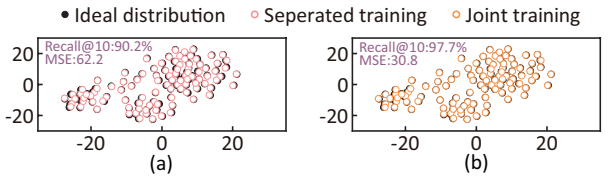

Fig. 3: t-SNE visualizations of separated training (a) and joint training (b) over the ideal distribution that is obtained from the float embedding model Resnet18 [42].

- **Index stage**: Each element in output O is related to M subspaces. As the query Q has been divided into M subspaces, each output value needs to select and accumulate M values from the intermediate results $\mathcal{IR}$. This selection process is based on the index table, which stores the indexes of values to accumulate in $\mathcal{IR}$.

D. Accuracy metrics

In neural vector search, the metric *recall* is frequently employed as a measure of retrieval accuracy. The term *recall* is often specified as *Recall@R* in such contexts, which is the average hit rate of queries for which the top 1 ground truth is ranked in the top R results. It is described as follows:

$$Recall_R = \frac{\sum_{i=1}^{n} hit_i}{n}, hit_i = \begin{cases} 1 & G_i \in \Psi_i \\ 0 & G_i \notin \Psi_i \end{cases} \quad (3)$$

n represents the number of retrievals, G_i equals the top 1 ground truth of the ith query, and Ψ_i is the top R results of the i-th query retrieved by the neural vector search.

III. Why We Need a Unified Neural Vector Search

The existing NVS solutions generally employ a separate construction scheme, which hinders the recall from reaching the global optimum and impacts search latency, energy consumption, hardware area, cost, and utilization. To prove this, we implement a typical NVS application, image search in Table VI, using Pytorch [40] and FAISS [25] on a server with a Xeon 4216 CPU and NVIDIA A100 GPU.

Unsatisfactory recall. In the existing NVS solutions, the training objective of a typical embedding model (e.g., BGE) is to distinguish the similarity of the semantics of objects as much as possible. However, a typical vector search algorithm in FAISS, PQ, aims to minimize reconstruction errors. Such inconsistent objectives of the two stages during training lead to sub-optimal recall. Fig. 3 shows 2-D visualizations of t-SNE (t-distributed Stochastic Neighbor Embedding) on randomly chosen images in the ImageNet dataset. It can be observed that joint training exhibits lower distortion error than the separated training scheme, improving Recall@10 to 97.7%.

Distinct computational and memory access pattern. The workload profiling results in Table I reveal that the vector search stage exhibits a higher number of misses per kilo-instruction (MPKI) of L2 and L3 caches than the embedding stage, which are incurred by the random memory access of the lookup operator. The memory accesses per operation in

TABLE I: Workload Characteristics on CPU.

	Embedding	Vector Search
L2 Cache MPKI	16.9	41.7
L3 Cache MPKI	0.3	10.5
DRAM Bytes per FLOPs	0.031	42.058
DRAM Energy per FLOPs	0.617	214.678

TABLE II: Latency, energy, area, and cost of existing solutions.

	Latency (us)	Energy (pJ)	Chip Area (mm^2)	Cost
PCB-level	119.95	3266.83	34.33	$$$$
Package-level	70.38	1372.84	34.33	$$$
Chip-level	9.756	302.64	>13.66	$$
NeuVSA	**0.256**	**119.79**	**13.66**	$

The latency is 8KB data transfer latency between two FPGAs [10].
The latency of NoC is estimated using [11].
The energy is the 8KB data transfer energy [9], [44].
The chip area is estimated based on results in Section. VII

Table I also confirm that the vector search stage involves intensive memory accesses per operation. In a nutshell, the embedding stage and vector search stage exhibit distinct patterns, where the former involves a regular pattern, bounded by computation, and the latter presents an irregular pattern, bounded by memory. The distinct computational and memory access patterns exhibited by the embedding and vector search stages constrain the feasibility of performing NVS efficiently using a single accelerator.

Terrible latency, energy, area, cost, and utilization. A strawman solution to simultaneously support both stages is to integrate existing embedding accelerators and vector search accelerators. Table II shows the latency, energy, area and cost of three levels of accelerator integration: PCB-level, package-level, and chip-level. Although chip-level integration using Chiplet or NoC shows lower latency, energy, area, and cost over PCIe- and package-level integration, the difference of each IP in architecture and dataflow limits their adaptability from one model/data to another and restricts resource-sharing between them, resulting hardware under-utilization. Therefore, building a unified NVS framework is crucial to providing efficient NVS services.

IV. Learned PQ-based unified NVS algorithm

A. Challenges

As a popular algorithm in vector search [25], PQ can replace multiplication with the lookup operator [45], providing a chance to build a unified NVS algorithm. Previous works focus on using PQ to improve the efficiency of embedding models or vector search. Despite the fact that employing existing PQ solutions can enable the deployment of NVS, three algorithm-level challenges still impede the creation of a unified NVS algorithm:

C-1: Sub-optimal mapping strategy. Converting diverse operators such as the Convolution (CONV) operator and the Fully Connected layer to GEMM and then mapping to PQ is a natural idea, which is employed in many works such as LUT-NN [45] and PQA [16]. However, this method introduces unnecessary memory footprints. For an example of convolving 3×3 weight with a 9×9 input tensor to produce a 7×7 output tensor and the stride size is 1, mapping it to GEMM using Im2Col (Image-to-Column [67]) first and then to PQ increases the memory footprint of the input tensor by 5.4×.

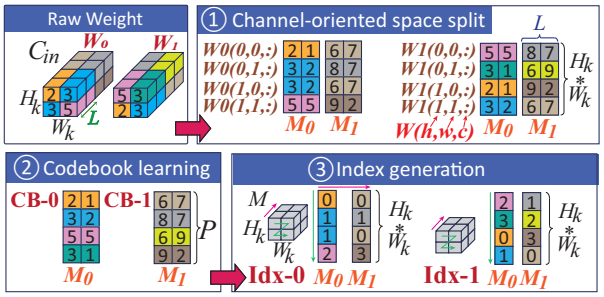

Fig. 4: Mapping the weight kernel of the CONV operator to PQ using a channel-oriented mapping. M_i refers the data in the same subspace related to the same codebook. CB-i and Idx-i represents the i-th codebook and i-th PQ index respectively.

C-2: Inconsistent algorithm. The existing PQ utilized in the embedding stage and the vector search stage follows different index and codebook generation strategies. For example, LUT-NN [45] and PQA [16] employ symmetric distance computation (SDC), where the index is generated online, and the lookup table is built offline. Conversely, FAISS [25] relies on asymmetric distance computation (ADC), where the index is generated offline and the lookup table is built online. This disparity impedes the construction of a unified NVS algorithm and may lead to increased hardware design complexity.

C-3: End-to-end joint training is challenging. The existing PQ training method aims to minimize the reconstruction error of the embedding stage or the vector search, which fails to support joint training. Therefore, how to design an effective loss function to enhance retrieval performance directly for improved recall is a tough problem. In addition, the index assignment steps in PQ that select optimum codewords c_p from the codebook $\mathcal{C}$ are not differentiable, which is also an obstacle to achieving end-to-end joint training.

B. Overview of learned PQ-based unified NVS algorithm

To overcome these challenges, we propose a learned PQ-based unified NVS algorithm. Specifically, to resolve ***C-1***, we introduce a channel-oriented mapping strategy to reduce memory overhead. To address ***C-2***, we design a PQ-based unified NVS algorithm that unifies two stages into the same computing paradigm. As for ***C-3***, we present an end-to-end joint training strategy with distortion-ranking loss and a codebook learning method for better recall.

1) Channel-oriented mapping strategy: To tackle the high memory overhead of the conventional mapping method, as shown in Fig. 4, we propose a channel-oriented mapping method that applies space split procedure directly in the direction of the channel instead of using Im2Col to unfold 3D tensor. Taking CONV as an example in Fig. 4, it also consists of three steps: (1) ***channel-oriented space split***:

Algorithm 1 ***Unified neural vector search framework***

Input: Input matrix $X \in \mathbb{R}^{M \times H_i \times W_i \times L}$; codebook $\mathcal{C} \in \mathbb{R}^{M \times P \times L}$; index $\mathcal{I} \in \mathbb{R}^{C_{out} \times H_k \times W_k \times M}$; bias $B \in \mathbb{R}^{C_{out}}$; product type $pt \in \{cosine, ip, l2\}$; stride s.
Output: Output matrix $O \in \mathbb{R}^{H_o \times W_o \times C_{out}}$.

1: **for** h_i to H_i **do** ▷ Input parallelism (IPL)
2: **for** w_i to W_i **do** ▷ Input parallelism (IPL)
3: $LUT \leftarrow$ ***Product***$(X[:, h_i, w_i, :], \mathcal{C}, pt)$
4: $O \leftarrow O +$ ***Indexing***$(\mathcal{I}, LUT, h_i, w_i, s)$
5: **return** $O = O + B$

Algorithm 2 ***Product***

Input: Input matrix $X \in \mathbb{R}^{M \times L}$; codebook $\mathcal{C} \in \mathbb{R}^{M \times P \times L}$; product type $pt \in \{cosine, ip, l2\}$.
Output: lookup_table $LUT \in \mathbb{R}^{M \times P}$.

1: **for** m to M **do** ▷ Inter-subspace parallelism (TPL)
2: **for** p to P **do** ▷ Centroid parallelism (CPL)
3: **for** l to L **do** ▷ Intra-subspace parallelism (SPL)
4: $LUT[m, p] += pt(X[m, h_i, w_i, l], \mathcal{C}[m, p, l])$
5: **return** LUT

Algorithm 3 ***Indexing***

Input: Index $\mathcal{I} \in \mathbb{R}^{C_{out} \times H_k \times W_k \times M}$, lookup_table $LUT \in \mathbb{R}^{M \times P}$, input height and width position h_i, w_i, stride s.
Output: Output matrix $O \in \mathbb{R}^{C_{out} \times H_o \times W_o}$.

1: **for** m to M **do** ▷ Inter-subspace parallelism (TPL)
2: **for** w_k to **range**$(w_i \ \% \ s, min(W_k, w_i + 1), s)$ **do**
3: **for** h_k to **range**$(h_i \ \% \ s, min(H_k, h_i + 1), s)$ **do**
4: $w_o = (w_i - w_k)/s; h_o = (h_i - h_k)/s$
5: **for** c_{out} to C_{out} **do** ▷ Output channel parallelism (OPL)
6: $pos \leftarrow \mathcal{I}[c_{out}, h_k, w_k, m]$
7: $O[c_{out}, h_o, w_o] \mathrel{+}= LUT[m, pos]$ ▷ Irregular Access
8: **return** O

The tensors X and W are each divided into M subspaces aligned with the dimension C_{in}, resulting in sub-vectors of dimension $L{=}C_{in}/M$. The m-th subspace of X and W is $X_{(m)} {\in} \mathbb{R}^{(H_k \times W_k) \times L}$ and $W_{(m)} {\in} \mathbb{R}^{(C_{out} \times H_k \times W_k) \times L}$, respectively. (2) ***codebook learning*** and (3) ***index generation***: m-th codebook $\mathcal{C}_{(m)} {\in} \mathbb{R}^{P \times L}$ with P codewords and the index $\mathcal{I}_{(m)} {\in} \{0, 1\}^{C_{out} \times H_k \times W_k \times P}$ for kernel W by optimized:

$$\min_{\mathcal{C}_{(m)}, \mathcal{I}_{(m)}} \left\| W_{(m)} - \mathcal{I}_{(m)} \mathcal{C}_{(m)} \right\|_2^2 \tag{4}$$

Note that the optimization function Eq. 4 is used to initialize the index $\mathcal{I}$ and the codebook $\mathcal{C}$ before launching the joint training strategy(Section. IV-B3). Table III shows that the benefits of channel-oriented mapping strategy improves with the growth of the kernel size over Im2Col used by LUT-NN [45].

2) PQ-based unified NVS algorithm: Despite the channel-oriented mapping strategy provides an efficient mapping method for NVS, the inconsistency between symmetric distance computation (SDC) and asymmetric distance computation (ADC) (***C-2 in Section. IV-A***) is still an obstruction to move towards a unified NVS. Supporting both ADC and SDC incurs additional hardware overhead. Considering that ADC exhibits lower quantization distortion compared to SDC [25].

TABLE III: The advantages of channel-oriented mapping.

Kernel Size	Computing complexity (GOPS)			Memory footprint (MB)		
	Our	Im2Col	Reduction	Our	Im2Col	Reduction
3	0.56	1.36	2.45×	12.27	108.31	8.83×
5	1.34	3.72	2.77×	12.28	295.44	24.05×
7	2.49	7.15	2.88×	12.31	568.58	46.20×
11	5.78	17.02	2.95×	12.38	1352.99	109.32×

Thereby, based on ADC, we propose a PQ-based unified NVS algorithm, which unifies two stages into the same computing and memory access paradigm. As shown in Alg. 1, the algorithm includes two stages:

① ***Product stage*** receives the input vectors in each subspace and applies them with each codeword in the codebook $\mathcal{C}$ to produce the lookup table LUT, depending on the chosen product type $pt \in \{cosine, ip, l2\}$ (line 4 in Alg. 2). When $pt = ip$, the product stage acts as MatMul. This is particularly important for the transformer [56] because the input matrices Q, K, and V of its attention operator are dynamically generated, which cannot be quantized using PQ. More importantly, this feature enables us to perform mixed-precision computing, where some operators use MatMul and other operators are mapped to PQ.

② ***Indexing stage*** fetches the pre-computed results from LUT based on the index $\mathcal{I}$, and then accumulated to generate the final result O (line 6-7 in Alg. 3). Besides, this stage can be used to approximate various non-linear functions by storing pre-calculated values in lookup-table.

3) End-to-end joint training strategy: Although the proposed PQ-based unified NVS algorithm achieves the unification of two stages, the absence of effective loss function and the non-differentiable of index assignment hinders the achievement of end-to-end training with joint optimization (***C-3 in Section. IV-A***). To deal with this dilemma, we propose an end-to-end joint training algorithm with two strategies:

Distortion-ranking loss. Combining LUT-NN and FAISS can follow a two-step training procedure. The first is to train the PQ-based embedding model with the MSE loss and then construct the codebook and index. However, the two-step method is task-independent and cannot benefit from supervised information. Thereby, we propose distortion-ranking loss to reduce the distortion error of PQ and accurately evaluate the retrieval quality. Specifically, unlike the prior approach that only focused on optimizing distortion error, we further use the pair-wise ranking loss in Eq. 5, a typical loss function in the information retrieval domain, to boost the final recall.

$$\mathcal{L}_{ranking} = \mathcal{F}(d(q, r_p), d(q, r_n)) \tag{5}$$

where $d(q, r)$ is the similar score between the query embedding q and the search result embeddings r. Meanwhile, r_p is a positive result similar to q, and r_n is a negative result dissimilar to q. The goal of Eq. 5 is to aggregate similar samples and disperse dissimilar samples. Meanwhile, to be closer to the practical situation, we leverage quantized query embedding $\hat{q}$ and result embeddings r to evaluate similar scores. Besides, we leverage the top-ranked negatives method

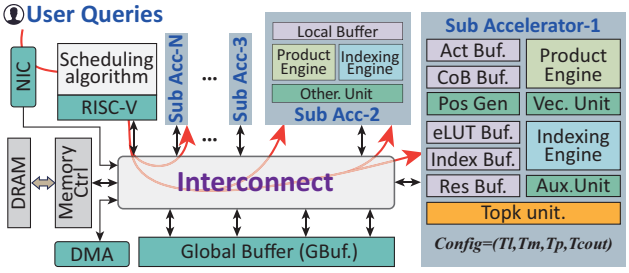

Fig. 5: The architecture of the NeuVSA accelerator.

proposed in [61] to construct negative samples S_q^- for each query q, and S_q^+ are the relevant labeled samples for each query q provided by the training dataset or generated using the brute-force method. In this case, the objective of the proposed distortion-ranking loss is formulated as follows:

$$\arg\min_{\mathcal{C}} \sum_{\hat{q}} \sum_{r_p \in S_q^+} \sum_{r_n \in S_q^-} \mathcal{F}(d(\hat{q}, r_p), d(\hat{q}, r_n)) + \|q - \hat{q}\|_2^2 \quad (6)$$

Codebook learning strategy is proposed to resolve the issue that the index assignments are non-differentiable. Considering that the codebook $\mathcal{C}$ is crucial for the reduction of distortion and the size of $\mathcal{C}$ is smaller, we opt to learn $\mathcal{C}$ instead of updating the index value. The codebook learning strategy first determines the value of the index based on the pre-trained model using K-means and freezes the value of the index before joint training. After that, we perform joint training to update the codewords c_p in the codebook $\mathcal{C}$.

V. The Proposed NeuVSA Accelerator

A. Challenges of NeuVSA accelerator

The proposed learned PQ-based unified NVS algorithm breaks the software-level separation. However, existing PQ accelerators such as PQA [16] fail to support the unified NVS as it adopts SDC. Worse still, deploying the unified NVS on existing PQ accelerators adopted ADC such as ANNA [29] is sub-optimal due to three architecture-level challenges:

C-4: Insufficient parallelization. As shown in Alg. 1, Alg. 2, and Alg. 3, the unified NVS algorithm offers six levels of parallelism by taking into account the batch-level parallelism. However, existing accelerators can exploit only up to four levels. For example, ANNA [29] lacks intra-subspace parallelism.

C-5: Frequent buffer access conflicts, When utilizing output parallelism, the unpredictable value of index $\mathcal{I} \in \{0,1\}^{N \times P}$ leads to significant random memory access and row conflicts in the on-chip buffer. Existing solutions either sacrifice output parallelism or adopt replication strategies [2], causing unnecessary memory footprint overhead.

C-6: Lack of scheduling. Two stages in NVS can exhibit distinct hardware resource demands when handling different workloads. However, the scheduling strategy provided by existing PQ accelerators focuses only on a single stage and lacks the scheduling mechanism required by NVS.

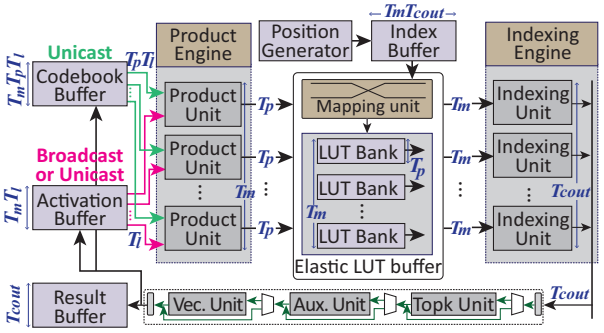

Fig. 6: The micro-architecture of sub-accelerator in NeuVSA.

B. Overview of the NeuVSA Architecture

On top of the unified NVS algorithm, we present NeuVSA, a unified and efficient homogeneous **Neu**ral **V**ector **S**earch **A**ccelerator. As shown in Fig. 5, to resolve ***C-4***, each sub-accelerator in NeuVSA contains a product engine and an indexing engine to exploit the inherent parallelism of PQ. they have on-chip dedicated buffers for activation data (**Act Buf.**), codebooks (**CoB Buf.**), indexes (**Index Buf.**), and outputs (**Res Buf.**). To overcome ***C-5***, we propose a structured index assignment strategy to regularize the index value and integrate an elastic LUT buffer (**eLUT Buf.**) on each sub-accelerator to flexibly adapt to changes in the size of the codebook C and the number of subspaces M. Meanwhile, **eLUT Buf.** is used to cache the intermediate results from the product engine. To overcome ***C-6***, we first design a homogeneous architecture with multiple sub-accelerators of varying configurations. After that, we propose a hardware-aware scheduling algorithm, which aims to coordinate task assignments across sub-accelerators based on the feature of user requests received from the network interface (NIC). The interconnect design between RISC-V cores and sub-accelerators uses an on-chip TileLink bus infrastructure. Since the primary focus of NeuVSA is on the accelerator architecture itself, detailed exploration of interconnect designs is beyond the scope of this paper.

C. The micro-architecture of the sub-accelerator

Fig. 6 shows the micro-architecture of the sub-accelerator. It contains two main engines: the product engine and the indexing engine. They are designed to boost the product stage and the indexing stage. Meanwhile, the position generator generates entry values based on the traversal trace on inputs, outputs, and weights. The generated entry values are used to access the index buffer and the result buffer. The Top-k unit tracks the k closest data and uses a hardware priority queue to achieve high throughput. The auxiliary unit (**Aux. Unit**) performs high-precision non-linear activation functions and data type cast operations. The vector unit (**Vec. Unit**) supports element-wise vector operations, including addition, subtraction, and multiplication of two vectors.

Product Engine aims to exploit four levels of parallelism, including inter-subspace parallelism (`TPL`), intra-subspace

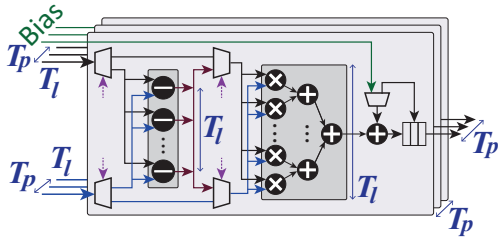

Fig. 7: The micro-architecture of product unit.

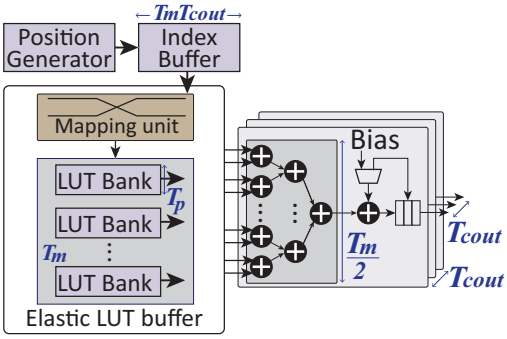

Fig. 8: The micro-architecture of indexing engine.

parallelism (SPL), centroid parallelism (CPL), and input parallelism (IPL). As shown in Fig. 6, the product engine contains T_m Product Units, each of which is responsible for one subspace to exploit TPL. Meanwhile, as shown in Fig. 7, each Product Unit consists of T_p lanes to exploit CPL, and each lane contains a tree-based multiplier-accumulators (MACs) that take vectors of T_l dimensions as input, aiming to exploit SPL. During execution, T_m input vectors with T_l length are sent to T_m product units, and T_m different vectors with T_l length from different centroids T_p are passed to each product unit. Each lane in the product unit processes one centroid. In this manner, $T_p \times T_m \times T_l$ multiplications are done in parallel. Besides, the product engine can exploit IPL when $T_m^{'}=T_m/T_{in}$ product units are grouped to handle one input tensor. In this way, T_{in} inputs and $T_m^{'}$ subspace can be done in parallel.

Position Generator. During the indexing stage, the positions of the accessed entries in the index buffer and the output buffer also need to be determined (lines 2-7 in Alg. 3). Since pre-computing these positions and saving them in memory requires extra memory footprints, the position generator is designed to generate access address based on the position of the output value and the raw computing flow of the input and weight matrix (lines 2-5 in Alg. 3).

Indexing Engine aims to aggregate the results produced from the product engine based on the index $\mathcal{I}$ and then accumulate them to produce the final output. The indexing stage aims to exploit TPL, IPL, and output channel parallelism (OPL). Thereby, as shown in Fig. 8, the indexing engine contains T_{cout} indexing units, each of which is in charge of one output value in the channel direction, to exploit OPL. Meanwhile, it can also be switched to exploit IPL when T_{cout}=1, where each indexing unit can generate one output

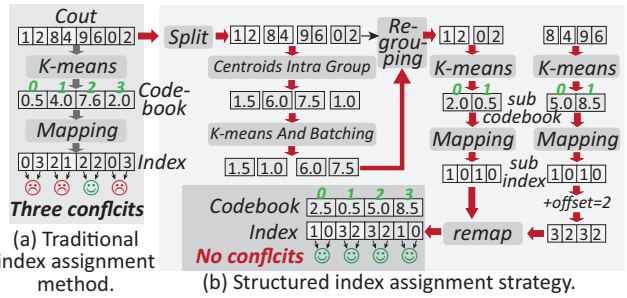

Fig. 9: An example of the structured index assignment. A code book is continuously stored in a bank with a width of T_p ($T_p = 2$ in this example) through folding. Accessing different rows of a bank in one cycle will cause conflicts.

value in H_i or W_i direction. The Position Generator issues the address for accessing entries in the index buffer.

D. Structured index assignment and elastic LUT buffer

When exploiting output parallelism OPL, multiple indexing units trying to access different rows of the same LUT bank can introduce row conflict, resulting in pipeline stall and decayed latency. To resolve this dilemma, at the algorithm level, we propose a structured index assignment strategy to allocate the value of the index with a dedicated constraint. At the architecture level, we design the elastic LUT buffer that supports flexible bank grouping to enhance the effect of the structured index assignment strategy.

Structured index assignment strategy. Traditional index assignment yields random indices, as shown in Fig. 9a, which can cause conflicts by accessing different rows of the same bank. To reduce these conflicts, we propose a structured index assignment strategy. This strategy initializes the codebook and the indices in groups, ensuring that adjacent indices indexed simultaneously access the same row.

As shown in Fig. 9a, when a matrix with C_{out} output dimensions arrives, we divide it into C_{out}/T_p groups, each of length T_p. We then compute the centroid for each group and apply k-means clustering, resulting in P/T_p clusters. Indices of groups within the same cluster are processed as a batch. This approach maintains the access order within each group while adjusting the order between groups, clustering similar groups for sub-codebook, and index initialization. Next, within each batch, we initialize sub-codebooks and indices as in traditional PQ. Finally, we concatenate the sub-codebooks to form the complete codebook for C_{out} and adjust the indices within each batch by adding offsets corresponding to their position in the complete codebook. The evaluation results in Section. VII-F show that this method avoids potential conflicts. The initialized codebook can be further refined through subsequent training.

Elastic LUT buffer. Despite the structured index assignment strategy that can avoid conflict from the algorithm aspect, its potential cannot be fully exploited by the rigid LUT buffer design due to the limited index range. For example, given a LUT bank with T_p width, once the row V is determined, the

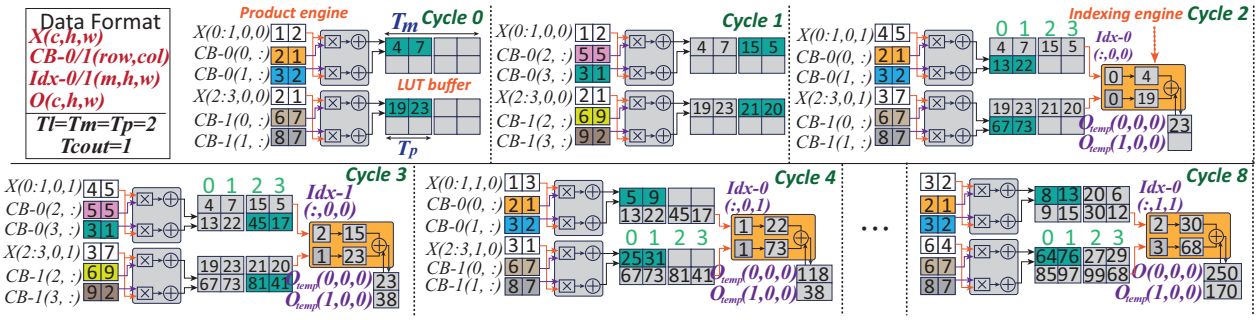

Fig. 10: The execution flow of the NeuVSA accelerator.

value index can only be located in $[V\times T_p : V\times T_p+T_p]$, which may damage the quality of the quantized model, especially when P is large. Besides, the rigid LUT buffer design cannot flexibly adapt to variants in the number of codewords P in codebook $\mathcal{C}$ and the number of subspace M, which may lead to hardware under-utilization. For example, when P is large while M is small, the rigid LUT buffer cannot allocate more LUT bank for codebook $\mathcal{C}$, resulting in more serious conflict and thus impacting hardware utilization. To this end, we propose the elastic LUT buffer. It can decompose the total T_m LUT bank into G groups equally so that each group contains T_m/G LUT banks and can provide $T_m\times T_p/G$ data pre-cycle. In this case, the index range is expanded to $[V\times T_m\times T_p/G : V\times T_m\times T_p/G+T_m\times T_p/G]$ when we select the same row V for each LUT bank in one group. In fact, we can select the different rows for different banks to achieve greater flexibility. Besides, when P is large while M is small, the elastic LUT buffer can allocate more resources for $\mathcal{C}$ to reduce conflict for extremely high utilization.

E. Example of the execution flow

We use the example in Fig. 11 to illustrate the behavior of NeuVSA at per cycle in Fig. 10, where NeuVSA is configured to T_m=2, T_p=2, T_l=2, and T_{cout}=1. In detail, at cycle 0, two $1\times1\times2$ input sub-vectors from subspace-0 $X(0{:}1,0,0)$ and subspace-1 $X(2{:}3,0,0)$ arrive at the product engine. Synchronously, each subspace of the codebook provides two centroid vectors, and the total four centroid vectors $CB\text{-}0/1(0{:}1,:)$ are sent to the product engine. In cycle 1, input sub-vectors remain unchanged while four new centroid vectors $CB\text{-}0/1(2{:}3,:)$ reach the product engine. At this point, the first pixel of the input tensor has traversed all centroid vectors in the codebooks, and the result is stored in the LUT buffer. Thereby, the index engine is triggered at cycle 2. The access position of the index table is up to the position of the accessed input pixel, which is provided by the position generator. As the first accessed pixel corresponds to the first weight $W0$, $Idx\text{-}0(:,0,0)$ is used to lookup the LUT buffer. After that, two values generated by the lookup operation are accumulated to produce an intermediate output result $O_{temp}(0,0,0)$. After that, $W1$ is processed in the following cycle 3. In addition, in cycle 2 and cycle 3, the product engine starts processing the two subvectors of the second pixel in the convolution window.

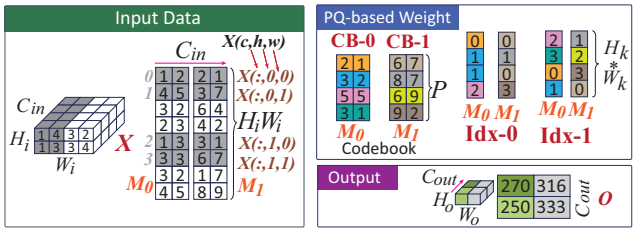

Fig. 11: An example of PQ-based Conv operator.

In subsequent cycles, the operations of the product engine are similar to cycles 0-4 providing new subvectors of input tensor in the convolution window and the following centroid vectors of codebook until all pixels in the convolution window are accessed (at cycle 7). Meanwhile, the index engine operates similarly to cycles 2-3 providing a new result in the LUT buffer until all pixels in the convolution window are accessed. In cycle 8, the first output pixel in the channel direction finishes computation, and the next convolution window starts.

VI. Scheduling Strategy on NeuVSA

NeuVSA is a homogeneous design that can support multiple search tasks with different workload patterns, which requires an appropriate scheduling strategy to ensure high hardware utilization for high throughput.

In the context of NVS applications, each operator can be classified into three types: P-type, I-type, and B-type. P-type operators are dominated by the product stage, I-type operators are dominated by the indexing stage, and B-type operators have a balanced workload. Correspondingly, sub-accelerators are also categorized into three types: P-type accelerators, with more hardware resources in the product engine (i.e. $T_p\times T_l > T_{cout}$); I-type accelerators, where the indexing engine is more powerful (i.e. $T_{cout} > T_p\times T_l$); and B-type accelerators, where both engines are balanced (i.e., $T_p\times T_l = T_{cout}$).

To align operators with sub-accelerators of the same type whenever possible, we propose hardware-aware scheduling for NeuVSA, as shown in Alg. 4, to optimize the hardware resource utilization of each sub-accelerator. The algorithm activates whenever there are idle sub-accelerators. It begins by calculating the computational loads of the product stage (CMP_P) and the indexing stage (CMP_I) for each executable operator task within the current time interval (in line 3). The

tasks' type is also determined in real-time based on these two metrics. Then, it makes the scheduling decision depending on tasks' and sub-accelerators' types (in lines 5-10) until all idle sub-accelerators are occupied.

Algorithm 4 Hardware-aware scheduling for NeuVSA

Input: List of operator tasks, List of idle sub-accelerators
Output: Scheduled operator tasks for idle sub-accelerators
1: **while** there are idle accelerators **do**
2: **for** each task op in executable tasks **do**
3: Calculate $CMP_P(op)$ and $CMP_I(op)$
4: **for** each idle sub-accelerator acc **do**
5: **if** acc is I-type **then**
6: Select task with $\max(CMP_I(t)/CMP_P(t))$
7: **else if** acc is P-type **then**
8: Select task with $\max(CMP_P(t)/CMP_I(t))$
9: **else if** acc is B-type **then**
10: Select task with $\min(|\log(CMP_P(t)) - \log(CMP_I(t))|)$
11: Assign task t to sub-accelerator acc

CMP_P and CMP_I are dynamically calculated based on the tasks' input parameters and PQ configurations. For example, consider three fully connected layer operators with C_{in}=64 and C_{out} values of 64, 128, and 256, respectively. They are configured with the same PQ parameters: P=32, M=16, and L=4. When the input tensor's batch size is 1, according to Alg. 2, the computational loads of the product stage of the three operators can be calculated with CMP_P=$M\times P\times L$=2048. For the operator with C_{out}=64, the computational load for the indexing stage, according to Alg. 3, is CMP_I=$M\times C_{out}$=1024<CMP_P. The operator is dominated by product stage, making it a P-type operator. For the operator with C_{out}=128, CMP_I=2048=CMP_P, depicts that the loads of both stages are balanced, making it a B-type operator. For the operator with C_{out}=256, CMP_I=4096>CMP_P, making it an I-type operator. Since the operator parameters and PQ configurations are provided by the user when initiating the query request, and the scheduler can retain them upon receiving the queries, the scheduler is allowed to dynamically determine the task type and schedule the tasks based on their parameters at each time interval.

VII. Evaluation

A. NeuVSA Implementation

As shown in Table IV, we implement a cycle-accurate simulator for performance evaluation, denoted as **NeuVSA**. The simulator models the micro-architectural behaviors of each module in NeuVSA and its timing behavior is co-verified with the RTL design. Meanwhile, the simulator also integrates Ramulator 2.0 [35] to simulate the memory access behavior of HBM2. We synthesize NeuVSA with Design Compiler under TSMC 45 nm GP library and estimate the power using PrimeTime PX. The area, power, and access latency of the on-chip SRAMs are estimated using CACTI [5].

We have also placed and routed the accelerator using the Synopsys IC Compiler II under 12nm process. The accelerator includes a single NeuVSA core and a RISC-V core used to send control signals and execute the scheduling algorithm.

TABLE IV: Accelerator baselines.

Target	Embedding		Vector Search	EMB&VS
Platform	PQA	DFX	ANNA	NeuVSA*
Frequency	800MHz			
Compute Unit	N_{out}=32 N_s=16 N_p=16 L_s=8	16×64 MFU 64 VFU	N_{cu}=256 N_u=128 N_{scm}=16	T_m=16 T_p=16 T_l=2 T_{cout}=32
On chip memory	3MB			
Off chip memory	HBM 2.0 with 460GB/s bandwidth			
Peak performance GOPS/sec	3686.4	1740.8	2252.8	1612.8
Power(W)	9.42	5.13	9.35	5.78
Area under 45nm processing node (mm^2)	18.92	14.48	20.39	13.66
*Act Buf:64KB, CoB Buf:2MB, eLUT Buf:128KB, Index Buf:512KB, Res Buf:256KB				

TABLE V: Area ratio under 12nm process.

	Area (mm^2)	Ratio
NeuVSA	0.4099	36.96%
SRAM cells of NeuVSA	0.3817	34.42%
Compute logic of NeuVSA	0.0264	2.38%
Control logic of NeuVSA	0.0018	0.16%
RISC-V core	0.6990	63.04%
Total	1.1089	100%

The Synopsys IC Compiler II determined that single-core NeuVSA occupies an area of 0.41mm^2 and consumes 1.93W of power under 12nm process. Table V depicts that the SRAM of NeuVSA accounts for 34.42% of the total area, and the total scheduling hardware logic area of the accelerator, including the area of RISC-V core for task scheduling and the area of control logic of NeuVSA for the data interaction between units, is 0.7mm^2, accounting for 63.2% of the total area.

B. Baselines

1) General Platform Baseline. We compare the system performance of NeuVSA to two general platforms, including 1) **CPU** platform, which is equipped with Xeon 4216 processor with 512GB DRAM; 2) **GPU** platform, which is equipped with NVIDIA A100 with 80GB GPU memory. The embedding model and the PQ-based vector search stage are implemented based on Pytorch [40] and Faiss [25], respectively.

2) Accelerator Baseline. As shown in Table IV, we use two typical neural network accelerators, DFX [19] and PQA [16] for the embedding stage. Note that PQA is a PQ-based neural network accelerator. Meanwhile, we select a typical PQ-based vector search accelerator ANNA [29] as the baseline for the vector search stage. For a fair comparison, we allocate their computation and on-chip memory resources the same as NeuVSA, executing under the same clock frequency and memory bandwidth, as indicated in Table IV, and evaluate their performance using the same method as NeuVSA.

To fully demonstrate the advantages brought by a unified architecture used by NeuVSA, we construct two types of baselines, **Separated baselines** that the embedding and

TABLE VI: Neural vector search applications.

Application	Embedding Model	Vector Search Datasets	Scale	Dim
Image search (IS) [42]	ResNet18	ImageNet	1281168	512
Document search (DS) [4]	DPR	Wiki_DPR	21015324	768
Text retrieval (TR) [12]	BGE-M3	MLDR	870114	1024

TABLE VII: Recall@10 and NDCG@10 loss.

	Recall@10			NDCG@10		
	Seperate training	Joint training	Loss decrease	Seperate training	Joint training	Loss decrease
IS	90.2%	97.7%	7.5%	66.0%	78.5%	12.5%
DS	91.1%	95.1%	4.0%	67.2%	76.8%	9.6%
TR	91.5%	94.7%	3.2%	68.4%	75.3%	6.9%

vector search stage perform on different specialized accelerators, including 3) DFX+ANNA and 4) PQA+ANNA. **Unified baselines** that the embedding and vector search stage performs on the same specialized accelerators, including 5) DFX and 6) ANNA. We deploy the vector search on DFX by converting the vector search to MatMul.

C. Neural vector search applications

As shown in Table VI, our evaluation covers three NVS applications, including one common application in the image domain (image search) and two prominent applications in the text domain (document search and text retrieval). In the embedding stage of the image search, we removed the classification layers of ResNet18 and fine-tuned them for embedding vector extraction. In the vector search stage, each application uses its embedding model to construct its embedding vector database on an application-specific dataset, including an image dataset (ImageNet [15]), a Wikipedia articles corpus (Wiki_DPR [27]), and a multilingual long-document retrieval dataset (MLDR [12]).

D. Metrics

Output quality metrics. We compared the output quality of the PQ-based NVS with the original NVS using two classical metrics, including Recall@R and normalized discounted cumulative gain [23] (NDCG@R). NDCG@R is a measure of ranking quality, which is described as follows:

$$NDCG_R = \sum_{i=1}^{R} \frac{rel_i}{log_2(i+1)} / \sum_{i=1}^{R} \frac{1}{log_2(i+1)} \qquad (7)$$

rel_i equals to whether the i-th element in top R ground-truth is in top R searching results.

Accelerator metrics. We evaluate NeuVSA and baselines in search latency, energy consumption, and throughput. Search latency refers to the time from query issuance to result arrival. Energy consumption indicates the energy utilized during request execution. Throughput is measured in queries per second (QPS) which indicates the number of requests per second.

TABLE VIII: Embedding model accuracy on GLUE.

Task	MNLI	QQP	QNLI	SST-2	CoLA	MRPC	RTE
Original	83.4%	71.2%	90.5%	93.5%	52.1%	88.9%	66.4%
LUT-NN	35.5%	63.2%	50.6%	49.3%	0.0%	31.6%	52.7%
Ours	79.9%	70.1%	87.4%	91.6%	50.8%	86.3%	64.5%

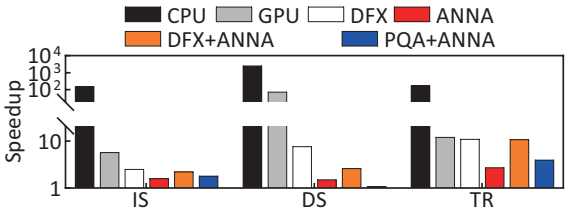

Fig. 12: Performance gain of NeuVSA over six baselines.

E. System Performance

Accuracy loss. We conduct an evaluation of the impact of joint training strategy on the output quality of NVS on three applications listed in Table VI. The results in Table VII reveal that compared to original NVS results, the joint training strategy achieves 95.8% and 76.8% in Recall@10 and NDCG@10, respectively, which is quite tolerable. When compared to the separated training strategy, the joint training strategy decreases the average losses of 4.6% and 9.4%, respectively, in Recall@10 and NDCG@10. This is because the joint training strategy considers the errors in the embedding stage and the searching stage uniformly and takes the recall of NVS as the optimization goal so that it can achieve better retrieval accuracy compared to the separated training strategy.

Furthermore, we evaluate the classification accuracy of the BERT-base [28] embedding model using LUT-NN and our algorithm on the GLUE [48] dataset. Table VIII shows our algorithm achieves higher accuracy on all tasks over LUT-NN [45], indicating that the codebook learning method can ensure low accuracy loss while reducing memory overhead.

Speedup. We evaluate the end-to-end latency of NeuVSA against six baselines using three applications in Table VI. Fig. 12 shows NeuVSA achieves 416.17× and 17.37× average speedup over CPU and GPU, respectively. This is because the PQ-based unified NVS algorithm used by NeuVSA can greatly reduce the amount of computation and memory access. Compared to separated baselines and unified baselines, NeuVSA achieves 2.82× and 3.35× speedup on average. This is primarily attributed to NeuVSA maintaining a much smaller lookup table than PQA and having additional data flow and caching optimizations for PQ-based neural network operators compared to ANNA, thereby reducing computation and memory access overheads. It is noted that the speedup of NeuVSA over the unified baseline is higher than that of the separated baseline. Although the unified baseline can improve resource utilization, applying these accelerators designed for a certain stage of NVS to the entire NVS process will instead bring greater overhead due to the mismatch of computing and memory access patterns.

Energy consumption. As illustrated in Fig. 13, NeuVSA

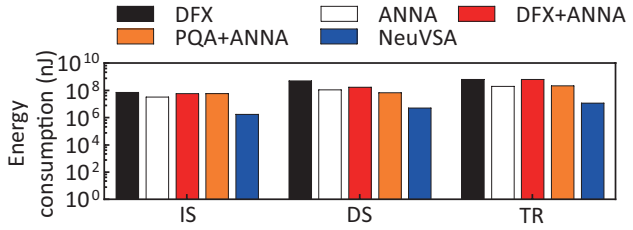

Fig. 13: Energy savings of NeuVSA over accelerator baselines.

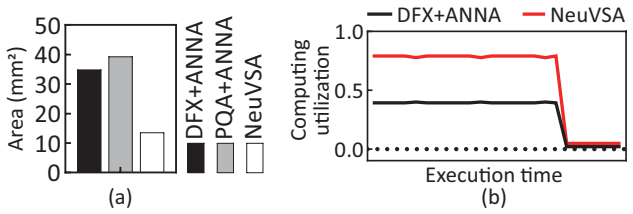

Fig. 14: (a) Area savings of NeuVSA. (b) Computing utilization on image search.

reduces energy consumption by factors of 28.35× and 34.10× compared to separated baselines and unified baselines, respectively. This is because off-chip memory access consumes most of the energy except for computing operations, which is particularly obvious for the PQA accelerator, as it requires reading large lookup tables from DRAM, resulting in significant energy consumption. Unlike PQA, NeuVSA achieves data compression using PQ and reduces the energy overhead incurred by data movement and computing.

Area savings. Fig. 14a shows the area of NeuVSA and baselines. It can be observed that NeuVSA reduces area overhead by 61% and 65% compared to simply combining DFX with ANNA and PQA with ANNA to support NVS, respectively. This is because the proposed unified framework allows one hardware to support two stages simultaneously instead of designing two completely different accelerators.

Utilization. Fig. 14b shows the utilization of NeuVSA surpasses the direct combination of existing accelerators designed for the embedding stage and the vector search stage. This is because NeuVSA provides a unified computing paradigm for NVS, which maximizes the utilization of hardware resources by allowing the embedding stage and the vector search stage to use the same set of hardware resources.

F. System Breakdown and Optimization

Speedup breakdown. We further analyze the speedup breakdown to identify the main sources of performance improvement. For the embedding stage, Fig. 15a depicts that NeuVSA achieves an average speedup of 28.93×, 7.58×, 5.56×, and 2.08× compared to CPU, GPU, DFX, and PQA on embedding stage, respectively. These benefits come from (a) PQ lessens the computational and data movement demands of neural networks on limited computing resource accelerator platforms, and (b) NeuVSA enhances the benefits of PQ by parallelizing computing and indexing operations. For the vector search

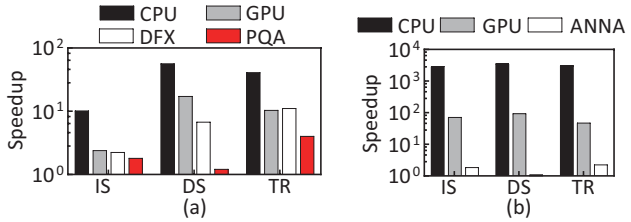

Fig. 15: The speedup breakdown of NeuVSA over baselines on (a) the embedding stage and (b) the vector search stage.

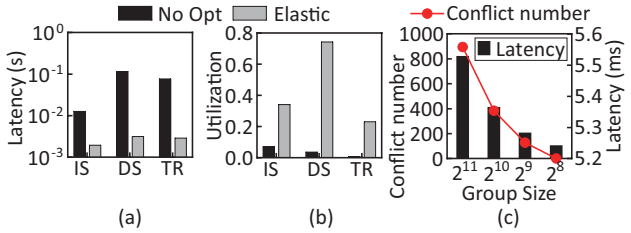

Fig. 16: (a) Latency of NeuVSA (No-Opt) and NeuVSA (Elastic). (b) Computing utilization. (c) The pros of structured index assignment.

stage, Fig. 15b demonstrates the remarkable performance improvement of NeuVSA with an average speedup of 3200.65×, 69.12×, and 1.65× compared to CPU, GPU, and ANNA, respectively. This is because NeuVSA can reuse the results of the product engine to eliminate unnecessary memory access and exploit inter-subspace parallelism that ANNA does not consider.

Elastic LUT buffer and structured index assignment. To analyze the effectiveness of the elastic LUT buffer, we constructed a baseline NeuVSA (`No-Opt`) without elastic on-chip buffer optimization and compared it with the optimized NeuVSA (`Elastic`). As shown in Fig. 16a, the elastic LUT buffer allows NeuVSA to achieve average speedups of 18.46×, respectively. This is because it eliminates buffer conflicts and pipeline stalls in the indexing stage, thus maintaining a high degree of computing parallelism and hardware resource utilization, which is proved in Fig. 16b.

When $P > T_m \times T_p$, even if elastic LUT buffer is applied, buffer conflicts will still occur, and structured index assignment is required to limit the group size within $T_m \times T_p$ to avoid conflicts. To verify the impact of group size on the number of conflicts and performance, we select a fully connected layer with a weight matrix size of 4096×4096 and perform product quantization with parameters P=2048 and L=2, where the group size is swept from 2048 (equivalent to not using structured index assignment) down to 256. As shown in Fig. 16c, as the group size decreases, structured index assignment can continuously reduce the number of conflicts and thus improve the performance of NeuVSA.

G. Sensitivity Analysis

PQ parameters. We examine two important factors in PQ: subspace length L and number of centroids P. We sweep L

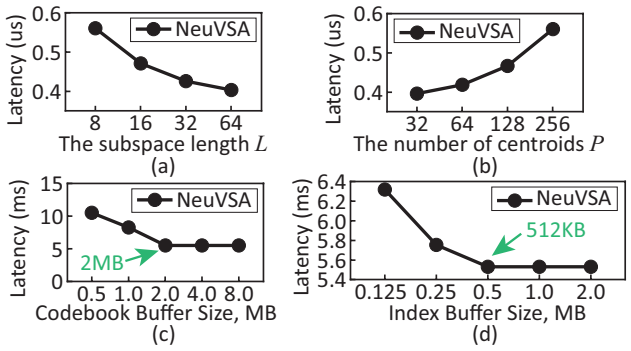

Fig. 17: Latency varies with (a) the subspace length L, (b) the number of centroids P, (c) the size of the codebook buffer (MB), and (d) the size of the index buffer (MB).

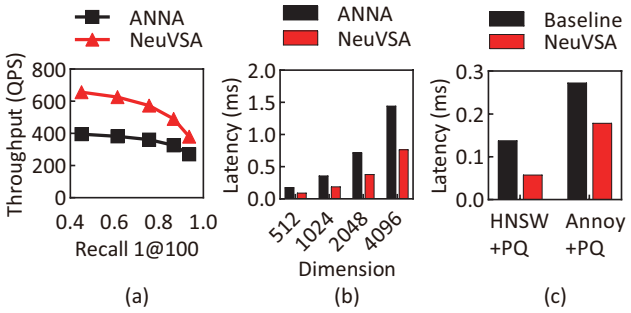

Fig. 18: (a) The throughput of NeuVSA and ANNA on billion-scale image search application. (b) The latency of NeuVSA and ANNA on various dimension vector search. (c) The latency of NeuVSA-accelerated HNSW+PQ and Annoy+PQ algorithms.

from 8 to 64 and P from 32 to 256. Fig. 17a shows increasing L can reduce the number of subspace M and the execution time of NeuVSA. This is because the T_m and T_p of NeuVSA in this experiment are configured to 1 and 256, which causes the benefits introduced by the decrease of M to be amortized over the increase of L. Thereby, the execution latency of the product stage remains unchanged. In contrast, in the indexing stage, the decrease of M means the subspaces that need to be traversed are decreased, which can achieve a reduction in total latency. Unlike L, raising P from 32 to 256 can increase latency, as shown in Fig. 17b. This is because increasing P introduces more computing and memory overhead in the product stage and does not affect the indexing stage.

Buffer size. We only analyze the impact of the Codebook buffer and Index buffer as the increase in other buffers has a negligible effect on the performance. As shown in Fig. 17c and d, the latency of NeuVSA gradually decreases with the increase in codebook buffer size from 0.5MB to 2MB until the codebook buffer size exceeds 2MB. A similar trend can be observed with the index buffer, where the curve's knee is at 0.5MB. This is because a small buffer results in more DRAM access, and increasing the buffer size has no effect once the balance between transmission and computation is achieved.

H. Scalability and Generalizability

Scalability. To validate the scalability of NeuVSA, we selected the representative image search application and compared the throughput of NeuVSA and ANNA on the billion-scale dataset. As shown in Fig. 18a, it can be observed that NeuVSA also achieves higher throughput than ANNA with the same recall on large-scale datasets. We also evaluate the latency of NeuVSA and ANNA on multiple vector datasets ranging from 512 to 4096 dimensions. To ensure that the accuracy does not change, we keep the codebook size P and the subspace size L constant. In this case, only the number of subspaces M changes, and it maintains a linear relationship with the vector dimension. As shown in Fig. 18b, NeuVSA achieves a higher performance than ANNA in high-dimensional vector search scenarios above a thousand dimensions. This is because the number of subspaces does not impact the effectiveness of optimization techniques used by NeuVSA, such as structured index assignment and elastic LUT buffer. Therefore, the speedup of NeuVSA over ANNA is not significantly affected as the dimensionality increases, which implies that NeuVSA can efficiently support high-dimensional vectors.

Generalizability. As vector search databases grow, distance computations emerge as a bottleneck for tree-based and graph-based approaches, prompting the adoption of hybrid indexing algorithms for acceleration. To validate NeuVSA's acceleration of hybrid indexing, we selected the graph-based HNSW [37] algorithm and the tree-based Annoy [8] algorithm and replaced the brute-force distance computations with PQ. We first evaluate these two hybrid indexing algorithms with the CPU platform as the baseline on the vector search stage of image search applications, whose scale and vector dimensions are shown in Table VI. Then, we offloaded the PQ distance computation to NeuVSA and evaluate this approach on the same dataset. Fig. 18c despicts that NeuVSA-accelerated HNSW+PQ and Annoy+PQ achieve speedups of 2.38× and 1.53×, respectively, compared to the baseline HNSW+PQ and Annoy+PQ. This is because NeuVSA can accelerate a large number of PQ distance computations inherent in searching processes, exhibiting a high degree of generalizability in hybrid indexing algorithms.

I. Hardware-aware Scheduling

Evaluation method. To evaluate the hardware-aware scheduling strategy, we first configure three accelerators with different parameters in Table IX for scheduling, which correspond to three accelerator types described in Section. VI, respectively. We also construct a baseline that maps all tasks of an application on accelerators based on the arrival order of tasks rather than the types of tasks and accelerators. Then, to simulate the real batch processing environment, we construct three scenarios: mixed-task batch consisting of 8 image search queries and 8 document search queries (8IS8DS), single-task batch consisting of 16 image search queries (16IS), and single-task batch consisting of 16 document search queries (16DS). Finally, to evaluate the ability of our scheduling algorithm to

TABLE IX: Accelerator configurations.

Accelerator ID	Accelerator type	Compute unit			
		T_m	T_p	T_l	T_{cout}
1	P	8	32	2	32
2	B	16	16	2	32
3	I	16	16	1	64

TABLE X: Candidate task configurations.

Task type	Task parameters
P	$H_i = W_i = 14, C_{in} = 256$ $H_k = W_k = 1, stride = 1, padding = 2$ $H_o = W_o = 7, C_{out} = 512, M = 64, P = 256$
B	$H_i = W_i = 14, C_{in} = 256$ $H_k = W_k = 3, stride = 2, padding = 1$ $H_o = W_o = 7, C_{out} = 512, M = 64, P = 256$
I	$H_i = W_i = 7, C_{in} = 512$ $H_k = W_k = 3, stride = 1, padding = 1$ $H_o = W_o = 7, C_{out} = 512, M = 128, P = 256$

TABLE XI: Workload compositions.

Distribution type	Workload Composition
P-dominant	40 P-type, 10 B-type, and 10 I-type tasks
B-dominant	10 P-type, 40 B-type, and 10 I-type tasks
I-dominant	10 P-type, 10 B-type, and 40 I-type tasks
Average	20 P-type, 20 B-type, and 20 I-type tasks

adapt to different workload distributions, we selected three representative layers from the IS application as candidate tasks, as shown in Table X. Using these candidate tasks, we constructed four types of workload composition, as presented in Table XI.

Evaluation results. Fig. 19a shows that our scheduling algorithm reduces total latency by 12.13%, 11.50%, and 4.23% over the baseline scheduling on 8IS8DS, 16IS, and 16DS, respectively. This is because our scheduling algorithm can flexibly schedule layers to the most suitable accelerators, thereby alleviating the computing bottlenecks. Fig. 19b illustrates that our scheduling algorithm reduces the total latency by 18.49%, 15.38%, 22.61%, and 32.68%, respectively, compared to the baseline under the four workload compositions: P-dominant, B-dominant, I-dominant, and Average. This demonstrates that our scheduling algorithm can adapt to various workload distributions and achieves the maximum optimization effect when the number of different task types is relatively balanced due to its ability to match tasks to the most suitable accelerator.

Throughput. We evaluate the throughput of NeuVSA and the general-purpose platform baseline in the 16IS and 16DS scenarios. As shown in Fig. 19c, NeuVSA achieves a higher throughput compared to both the CPU and the GPU. This is mainly because 1) NeuVSA achieves a higher speedup on a single core through the PQ-based unified algorithm, which is particularly prominent in the searching stage of each task in the 16DS scenario. 2) The hardware-aware scheduling strategy improves the utilization of hardware resources, thereby improving the parallelism of task execution.

VIII. Related Work

There are many software frameworks for the embedding stage [1], [40], [55] and the vector search stage [25], [30], [50],

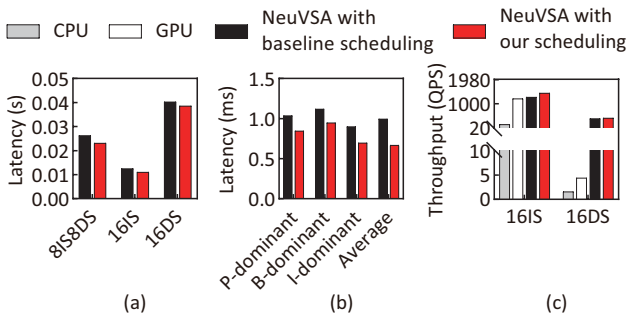

Fig. 19: (a) Latency comparison on various application compositions. (b) Latency comparison on various workload compositions. (c) Throughput comparison.

[54], [64], [65]. Traditional platforms such as CPU and GPU cannot satisfy the demands of neural vector search. There is a need for a specialized accelerator to optimize the embedding stage and the vector search stage.

Hardware acceleration of Embedding: The embedding stage of neural networks can employ a range of architectures, from conventional ones like CNN [6], RNN [43], and GNN [57] to newer ones like transformer [56]. To improve the performance of this stage, previous work has focused on accelerating these neural networks using tools like TPU [26], Eyeriss [13], HyGCN [58], DFX [19], and SpAtten [49]. However, the vector search stage of neural vector search presents new opportunities for parallelism and data reuse patterns that cannot be exploited by these accelerators. While [16] [63], and [53] have customized accelerators based on PQ algorithms for neural networks, they are not efficient for vector search due to the lack of necessary components like top-k.

Hardware acceleration of Vector Search: A lot of specific accelerators are proposed to boost quantization-based vector search. For example, [2] and [62] deploy PQ-based vectors on FPGA from the algorithm and architecture optimization, respectively. FANNS [24] is a scalable FPGA vector search framework that co-designs hardware and algorithm. Due to the limited performance of the FPGA platform, ANNA [29] proposes a fixed ASIC design for PQ supporting arbitrary algorithm parameters. There are still many vector search designs that are not based on PQ algorithms, which mainly focus on exact vector search such as Gemini APU [46] and FAERY [60] and graph-based vector search such as CXL-ANNS [22] and VStore [32], resulting in the inability to effectively support over billion-scale vector search.

Hardware acceleration of Neural Vector Search: TPU-KNN [14] focuses on distance computation for k-NN on TPUs using the roofline model. However, it still suffers from high search latency due to the curse of high dimensionality in exhaustive vector search. Even though Cognitive SSD [31] and DeepStore [36] can enable intelligent data search, they are optimized for storage systems and adopt a separate architecture. In contrast, NeuVSA has a unified architecture that can also be used in storage systems.

IX. Conclusion and Future Work

NVS is a critical component in information retrieval systems. However, current solutions are not designed for both the embedding stage and vector search stage simultaneously, leading to decayed recall, high latency, area, energy and cost. To this end, we introduce NeuVSA, a unified and efficient NVS accelerator based on algorithm and architecture co-design. It adopts joint training, unified architecture, and scheduling to boost the performance of NVS. Experiments show that NeuVSA can increase accuracy and reduce latency, energy, and area compared to existing solutions.

Concurrent with the demands imposed by the growth of the service and user scale, high concurrency emerges as a pivotal requirement for NVS systems. To address this, we decide to implement more agile data flow mechanisms and advanced scheduling algorithms to support the management and allocation of unified hardware resources across sub-accelerators in the future, which can enhance resource utilization efficiency and granularity, thereby solving the evolving challenges in the realm of large-scale and high concurrency NVS system. We further intend to implement our accelerator utilizing 3D stacking technologies and more advanced process nodes, thereby pushing to minimize the latency, footprint, and energy consumption of the accelerator, enhancing its overall performance and efficiency.

Acknowledgment

We thank the reviewers for their comments and are particularly grateful to our shepherd for his/her feedback. This work is supported in part by the Strategic Priority Research Program of the Chinese Academy of Sciences (under Grants No. XDB0660100), the National Key R&D Program of China (under Grants No. 2023YFB4404400), the Natural Science Foundation of China (under Grants No. 62202453, 62302283, 62332021, 62472007, 62222411, 62072333), and the State Key Lab of Processors, Institute of Computing Technology, CAS (under Grant No. CLQ202309, CARCH6102), as well as supports from Huawei Technologies. Dr. Shengwen Liang, Dr. Ying Wang, and Dr. Renhai Chen are the corresponding authors.

References

[1] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard *et al.*, "{TensorFlow}: a system for {Large-Scale} machine learning," in *12th USENIX symposium on operating systems design and implementation (OSDI 16)*, 2016, pp. 265–283.

[2] A. M. Abdelhadi, C.-S. Bouganis, and G. A. Constantinides, "Accelerated approximate nearest neighbors search through hierarchical product quantization," in *2019 International Conference on Field-Programmable Technology (ICFPT)*, 2019, pp. 90–98.

[3] Alibaba, "Alibaba proxima." [Online]. Available: https://github.com/alibaba/proxima

[4] S. Althammer, S. Hofstätter, M. Sertkan, S. Verberne, and A. Hanbury, "Parm: A paragraph aggregation retrieval model for dense document-to-document retrieval," in *European Conference on Information Retrieval*. Springer, 2022, pp. 19–34.

[5] R. Balasubramonian, A. B. Kahng, N. Muralimanohar, A. Shafiee, and V. Srinivas, "Cacti 7: New tools for interconnect exploration in innovative off-chip memories," *ACM Trans. Archit. Code Optim.*, vol. 14, no. 2, jun 2017. [Online]. Available: https://doi.org/10.1145/3085572

[6] P. Ballester and R. Araujo, "On the performance of googlenet and alexnet applied to sketches," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 30, no. 1, 2016.

[7] O. Barkan and N. Koenigstein, "Item2vec: neural item embedding for collaborative filtering," in *2016 IEEE 26th International Workshop on Machine Learning for Signal Processing (MLSP)*. IEEE, 2016, pp. 1–6.

[8] E. Bernhardsson, "Spotify annoy." [Online]. Available: https://github.com/spotify/annoy

[9] M. Bichan, C. Ting, B. Zand, J. Wang, R. Shulyzki, J. Guthrie, K. Tyshchenko, J. Zhao, A. Parsafar, E. Liu, A. Vatankhahghadim, S. Sharifian, A. Tyshchenko, M. De Vita, S. Rubab, S. Iyer, F. Spagna, and N. Dolev, "A 32gb/s nrz 37db serdes in 10nm cmos to support pci express gen 5 protocol," in *2020 IEEE Custom Integrated Circuits Conference (CICC)*, 2020, pp. 1–4.

[10] Bittware, "Bittware 250 soc card." [Online]. Available: https://www.bittware.com/zh/products/250-soc/

[11] C.-H. O. Chen, S. Park, T. Krishna, S. Subramanian, A. P. Chandrakasan, and L.-S. Peh, "Smart: A single-cycle reconfigurable noc for soc applications," in *2013 Design, Automation Test in Europe Conference Exhibition (DATE)*, 2013, pp. 338–343.

[12] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," 2024.

[13] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE journal of solid-state circuits*, vol. 52, no. 1, pp. 127–138, 2016.

[14] F. Chern, B. Hechtman, A. Davis, R. Guo, D. Majnemer, and S. Kumar, "Tpu-knn: K nearest neighbor search at peak flop/s," *Advances in Neural Information Processing Systems*, vol. 35, pp. 15 489–15 501, 2022.

[15] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *2009 IEEE Conference on Computer Vision and Pattern Recognition*, 2009, pp. 248–255.

[16] J. Fernandez-Marques, A. F. AbouElhamayed, N. D. Lane, and M. S. Abdelfattah, "Are we there yet? product quantization and its hardware acceleration," *arXiv preprint arXiv:2305.18334*, 2023.

[17] Google, "Google vertex ai." [Online]. Available: https://cloud.google.com/vertex-ai?hl=zh-cn

[18] M. Grohe, "Word2vec, node2vec, graph2vec, x2vec: Towards a theory of vector embeddings of structured data," in *Proceedings of the 39th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*, ser. PODS'20. New York, NY, USA: Association for Computing Machinery, 2020, p. 1–16. [Online]. Available: https://doi.org/10.1145/3375395.3387641

[19] S. Hong, S. Moon, J. Kim, S. Lee, M. Kim, D. Lee, and J.-Y. Kim, "Dfx: A low-latency multi-fpga appliance for accelerating transformer-based text generation," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 616–630.

[20] L. Hu and S. Nooshabadi, "High-dimensional image descriptor matching using highly parallel kd-tree construction and approximate nearest neighbor search," *Journal of Parallel and Distributed Computing*, vol. 132, pp. 127–140, 2019.

[21] J.-T. Huang, A. Sharma, S. Sun, L. Xia, D. Zhang, P. Pronin, J. Padmanabhan, G. Ottaviano, and L. Yang, "Embedding-based retrieval in facebook search," in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2020, pp. 2553–2561.

[22] J. Jang, H. Choi, H. Bae, S. Lee, M. Kwon, and M. Jung, "CXL-ANNS: Software-Hardware collaborative memory disaggregation and computation for Billion-Scale approximate nearest neighbor search," in *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. Boston, MA: USENIX Association, Jul. 2023, pp. 585–600. [Online]. Available: https://www.usenix.org/conference/atc23/presentation/jang

[23] K. Järvelin and J. Kekäläinen, "Cumulated gain-based evaluation of ir techniques," *ACM Trans. Inf. Syst.*, vol. 20, no. 4, p. 422–446, oct 2002. [Online]. Available: https://doi.org/10.1145/582415.582418

[24] W. Jiang, S. Li, Y. Zhu, J. d. F. Licht, Z. He, R. Shi, C. Renggli, S. Zhang, T. Rekatsinas, T. Hoefler *et al.*, "Co-design hardware and algorithm for vector search," *arXiv preprint arXiv:2306.11182*, 2023.

[25] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with gpus," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2019.

[26] N. Jouppi, G. Kurian, S. Li, P. Ma, R. Nagarajan, L. Nai, N. Patil, S. Subramanian, A. Swing, B. Towles *et al.*, "Tpu v4: An optically

reconfigurable supercomputer for machine learning with hardware support for embeddings," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–14.

[27] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Online: Association for Computational Linguistics, Nov. 2020, pp. 6769–6781. [Online]. Available: https://www.aclweb.org/anthology/2020.emnlp-main.550

[28] J. D. M.-W. C. Kenton and L. K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of NAACL-HLT*, 2019, pp. 4171–4186.

[29] Y. Lee, H. Choi, S. Min, H. Lee, S. Beak, D. Jeong, J. W. Lee, and T. J. Ham, "Anna: Specialized architecture for approximate nearest neighbor search," in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2022, pp. 169–183.

[30] J. Li, H. Liu, C. Gui, J. Chen, Z. Ni, N. Wang, and Y. Chen, "The design and implementation of a real time visual search system on jd e-commerce platform," in *Proceedings of the 19th International Middleware Conference Industry*, ser. Middleware '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 9–16. [Online]. Available: https://doi.org/10.1145/3284028.3284030

[31] S. Liang, Y. Wang, Y. Lu, Z. Yang, H. Li, and X. Li, "Cognitive SSD: A deep learning engine for In-Storage data retrieval," in *2019 USENIX Annual Technical Conference (USENIX ATC 19)*. Renton, WA: USENIX Association, Jul. 2019, pp. 395–410. [Online]. Available: https://www.usenix.org/conference/atc19/presentation/liang

[32] S. Liang, Y. Wang, Z. Yuan, C. Liu, H. Li, and X. Li, "Vstore: In-storage graph based vector search accelerator," in *Proceedings of the 59th ACM/IEEE Design Automation Conference*, ser. DAC '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 997–1002. [Online]. Available: https://doi.org/10.1145/3489517.3530560

[33] P. Liu, S. Wang, X. Wang, W. Ye, and S. Zhang, "Quadrupletbert: An efficient model for embedding-based large-scale retrieval," in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2021, pp. 3734–3739.

[34] W. Liu, H. Wang, Y. Zhang, W. Wang, L. Qin, and X. Lin, "Ei-lsh: An early-termination driven i/o efficient incremental c-approximate nearest neighbor search," *The VLDB Journal*, vol. 30, pp. 215–235, 2021.

[35] H. Luo, Y. C. Tuğrul, F. N. Bostancı, A. Olgun, A. G. Yağlıkçı, and O. Mutlu, "Ramulator 2.0: A modern, modular, and extensible dram simulator," *IEEE Computer Architecture Letters*, vol. 23, no. 1, pp. 112–116, 2024.

[36] V. S. Mailthody, Z. Qureshi, W. Liang, Z. Feng, S. G. de Gonzalo, Y. Li, H. Franke, J. Xiong, J. Huang, and W.-m. Hwu, "Deepstore: In-storage acceleration for intelligent queries," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '52. New York, NY, USA: Association for Computing Machinery, 2019, p. 224–238. [Online]. Available: https://doi.org/10.1145/3352460.3358320

[37] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE transactions on pattern analysis and machine intelligence*, vol. 42, no. 4, pp. 824–836, 2018.

[38] I. Olonetsky and S. Avidan, "Treecann-kd tree coherence approximate nearest neighbor algorithm," in *Computer Vision–ECCV 2012: 12th European Conference on Computer Vision, Florence, Italy, October 7-13, 2012, Proceedings, Part IV 12*. Springer, 2012, pp. 602–615.

[39] Z. Pan, L. Wang, Y. Wang, and Y. Liu, "Product quantization with dual codebooks for approximate nearest neighbor search," *Neurocomputing*, vol. 401, pp. 59–68, 2020.

[40] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, "Pytorch: An imperative style, high-performance deep learning library," *Advances in neural information processing systems*, vol. 32, 2019.

[41] Y. Qiu, C. Zhao, H. Zhang, J. Zhuo, T. Li, X. Zhang, S. Wang, S. Xu, B. Long, and W.-Y. Yang, "Pre-training tasks for user intent detection and embedding retrieval in e-commerce search," in *Proceedings of the 31st ACM International Conference on Information Knowledge Management*, ser. CIKM '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 4424–4428. [Online]. Available: https://doi.org/10.1145/3511808.3557670

[42] L. N. Rani and Y. Yuhandri, "Similarity measurement on logo image using cbir (content base image retrieval) and cnn resnet-18 architecture," in *2023 International Conference on Computer Science, Information Technology and Engineering (ICCoSITE)*, 2023, pp. 228–233.

[43] A. Sherstinsky, "Fundamentals of recurrent neural network (rnn) and long short-term memory (lstm) network," *Physica D: Nonlinear Phenomena*, vol. 404, p. 132306, 2020.

[44] Z. Tan, H. Cai, R. Dong, and K. Ma, "Nn-baton: Dnn workload orchestration and chiplet granularity exploration for multichip accelerators," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021, pp. 1013–1026.

[45] X. Tang, Y. Wang, T. Cao, L. L. Zhang, Q. Chen, D. Cai, Y. Liu, and M. Yang, "Lut-nn: Empower efficient neural network inference with centroid learning and table lookup," in *Proceedings of the 29th Annual International Conference on Mobile Computing and Networking*, ser. ACM MobiCom '23. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: https://doi.org/10.1145/3570361.3613285

[46] G. Technology, "Gemini apu: Enabling high performance billionscale similarity search."

[47] K. Terasawa and Y. Tanaka, "Spherical lsh for approximate nearest neighbor search on unit hypersphere," in *Algorithms and Data Structures: 10th International Workshop, WADS 2007, Halifax, Canada, August 15-17, 2007. Proceedings 10*. Springer, 2007, pp. 27–38.

[48] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. Bowman, "GLUE: A multi-task benchmark and analysis platform for natural language understanding," in *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, T. Linzen, G. Chrupała, and A. Alishahi, Eds. Brussels, Belgium: Association for Computational Linguistics, Nov. 2018, pp. 353–355. [Online]. Available: https://aclanthology.org/W18-5446

[49] H. Wang, Z. Zhang, and S. Han, "Spatten: Efficient sparse attention architecture with cascade token and head pruning," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 97–110.

[50] J. Wang, X. Yi, R. Guo, H. Jin, P. Xu, S. Li, X. Wang, X. Guo, C. Li, X. Xu *et al.*, "Milvus: A purpose-built vector data management system," in *Proceedings of the 2021 International Conference on Management of Data*, 2021, pp. 2614–2627.

[51] M. Wang, X. Xu, Q. Yue, and Y. Wang, "A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search," *Proc. VLDB Endow.*, vol. 14, no. 11, p. 1964–1978, jul 2021. [Online]. Available: https://doi.org/10.14778/3476249.3476255

[52] S. Wang and R. Koopman, "Semantic embedding for information retrieval." in *BIR@ ECIR*, 2017, pp. 122–132.

[53] Y. Wang, S. Liang, H. Li, and X. Li, "A none-sparse inference accelerator that distills and reuses the computation redundancy in cnns," in *2019 56th ACM/IEEE Design Automation Conference (DAC)*, 2019, pp. 1–6.

[54] C. Wei, B. Wu, S. Wang, R. Lou, C. Zhan, F. Li, and Y. Cai, "Analyticdb-v: A hybrid analytical engine towards query fusion for structured and unstructured data," *Proc. VLDB Endow.*, vol. 13, no. 12, p. 3152–3165, aug 2020. [Online]. Available: https://doi.org/10.14778/3415478.3415541

[55] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz *et al.*, "Huggingface's transformers: State-of-the-art natural language processing," *arXiv preprint arXiv:1910.03771*, 2019.

[56] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz *et al.*, "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*, 2020, pp. 38–45.

[57] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and S. Y. Philip, "A comprehensive survey on graph neural networks," *IEEE transactions on neural networks and learning systems*, vol. 32, no. 1, pp. 4–24, 2020.

[58] M. Yan, L. Deng, X. Hu, L. Liang, Y. Feng, X. Ye, Z. Zhang, D. Fan, and Y. Xie, "Hygcn: A gcn accelerator with hybrid architecture," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2020, pp. 15–29.

[59] J. Yuan and X. Liu, "Product tree quantization for approximate nearest neighbor search," in *2015 IEEE International Conference on Image Processing (ICIP)*. IEEE, 2015, pp. 2035–2039.

[60] C. Zeng, L. Luo, Q. Ning, Y. Han, Y. Jiang, D. Tang, Z. Wang, K. Chen, and C. Guo, "{FAERY}: An {FPGA-accelerated} embedding-based

retrieval system," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 841–856.

[61] J. Zhan, J. Mao, Y. Liu, J. Guo, M. Zhang, and S. Ma, "Jointly optimizing query encoder and product quantization to improve retrieval performance," in *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*, 2021, pp. 2487–2496.

[62] J. Zhang, S. Khoram, and J. Li, "Efficient large-scale approximate nearest neighbor search on opencl fpga," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 4924–4932.

[63] J. Zhang and J. Li, "Pq-cnn: accelerating product quantized convolutional neural network on fpga," in *2018 IEEE 26th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 2018, pp. 207–207.

[64] Q. Zhang, S. Xu, Q. Chen, G. Sui, J. Xie, Z. Cai, Y. Chen, Y. He, Y. Yang, F. Yang, M. Yang, and L. Zhou, "VBASE: Unifying online vector similarity search and relational queries via relaxed monotonicity," in *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. Boston, MA: USENIX Association, Jul. 2023, pp. 377–395. [Online]. Available: https://www.usenix.org/conference/osdi23/presentation/zhang-qianxi

[65] Z. Zhang, C. Jin, L. Tang, X. Liu, and X. Jin, "Fast, approximate vector queries on very large unstructured datasets," in *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. Boston, MA: USENIX Association, Apr. 2023, pp. 995–1011. [Online]. Available: https://www.usenix.org/conference/nsdi23/presentation/zhang-zili

[66] X. Zhao, Y. Tian, K. Huang, B. Zheng, and X. Zhou, "Towards efficient index construction and approximate nearest neighbor search in high-dimensional spaces," *Proceedings of the VLDB Endowment*, vol. 16, no. 8, pp. 1979–1991, 2023.

[67] Y. Zhou, M. Yang, C. Guo, J. Leng, Y. Liang, Q. Chen, M. Guo, and Y. Zhu, "Characterizing and demystifying the implicit convolution algorithm on commercial matrix-multiplication accelerators," in *2021 IEEE International Symposium on Workload Characterization (IISWC)*. Los Alamitos, CA, USA: IEEE Computer Society, nov 2021, pp. 214–225. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/IISWC53511.2021.00029